Part 16

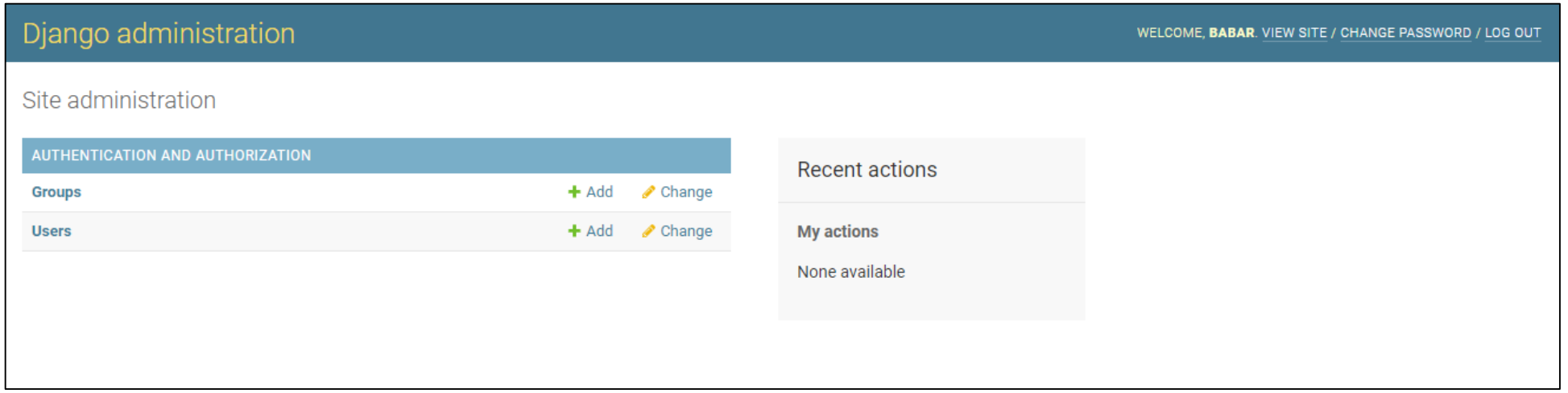
Django Admin Customization

VERSION: 4.0.1

WRITTEN AND DESIGN BY: BABAR ALI

Django Admin

Django Admin is one of the most important tools of Django. It's a full-fledged application with all the utilities a developer need. Django Admin's task is to provide an interface to the admin of the web project. Django's Docs clearly state that Django Admin is not made for frontend work.



Explanation

Django Admin's aim is to provide a user interface for performing CRUD operations on user models and other functionalities like authentication, user access levels, etc. These all are functions that make admin more productive. We have a separate tutorial, which explains these functions.

This tutorial on Django Admin customization is aimed at more advanced users. It will provide you knowledge of how the admin interface is customizable, how we can add and change particular things in the admin. This modification may make it more productive according to the use-case.

Django Admin Customization

We will be customizing some parts of the Django Admin. This customization will provide us with a better representation of objects. To implement it in your project, make a new app in your Django project named **products**.

- ✓ Create app for project name is "**Product**".
- ✓ Installed the Apps in "**settings.py**" file.
- ✓ Create Model for Product.
- ✓ Register you model in the "**admin.py**" file.
- ✓ Create Superuser for access the Admin Interface.

1. Register/ Unregister models from admin

A valid point of the argument is that you simply don't register the model. Yes, that's true. What if we want to remove default models like Groups, Users from admin? The answer is **unregister method**.

```
from django.contrib import admin
from django.contrib.auth.models import Group
from .models import Product

# Register the model
admin.site.register(Product)

# Unregister the model
admin.site.unregister(Group)
```

2. Changing the title of Django Admin

There are so many parts that can be changed from the front-end perspective. Like you can change the title written in the top-left corner of the web-page. **Django administration** can be changed to anything you like.

```
# Changing the admin header
admin.site.site_header = "DataFlair Django Tutorials"
```

3. ModelAdmin Class

The ModelAdmin class is a representation of user-defined models in the admin panel. It can be used to override various actions. There is a whole range of options opened with **ModelAdmin** Class.

```
# ModelAdmin Class
class ProductA(admin.ModelAdmin):
    exclude = ('description', )
```

We simply make the ModelAdmin class. This class is used to override the default class views. These default views are those which are created by an admin. We can think of the ModelAdmin class which is always generated by an admin.

4. Customizing List Display in Django Admin

You should add some products before we check out this one. Populate your database with 5-6 products. After you have added some products, open the products model from the admin panel. There you will see a list of Product objects. It only shows the names of the products like in the image.

```
# show the name, description in the admin page
list_display = ('name', 'description')
```

The **list_display** variable takes a list or tuple as input. You can provide the names of fields you want it to display.

5. Adding a filter in admin

How easy it would be to be able to filter objects. When dealing with a large database, it becomes necessary to see selected information. To perform that task easier, we can add a filter in the list view of the admin panel.

```
# filter by date
list_filter = ('mfg_date', )
```

Here, we simply mentioned the fields we wanted a filter for. You can add multiple fields at a time in the **list_filter** variable. It takes in the list as input. After adding the field name, the admin list_view changes to this.

Django Admin Actions

The admin actions are functions that can modify collective data easily. In this image below, deleting multiple objects is fairly easy. That is the utility of actions. Not just that, actions can do much more. We will also make some actions of our own.

Now, suppose we have a rating system for our products. We have three kinds of ratings, bad, good and excellent.

We are first going to edit **products/models.py** file. Add this code according to the image.

Django Model

```
from django.db import models

Rating = [
    ('b', 'Bad'),
    ('a', 'Average'),
    ('e', 'Excellent')
]

#DataFlair
class Product(models.Model):
    name = models.CharField(max_length = 200)
    description = models.TextField()
    rating = models.CharField(max_length = 1, choices = Rating)

    def show_desc(self):
        return self.description[:50]
```

Django Admin Customization

```
from django.contrib import admin
from django.contrib.auth.models import Group
from .models import Product

# Admin Action Functions
def change_rating(modeladmin, request, queryset):
    queryset.update(rating = 'e')

# Action description
change_rating.short_description = "Mark Selected Products as Excellent"

# ModelAdmin Class # DataFlair
class ProductA(admin.ModelAdmin):
    list_display = ('name', 'description', 'mfg_date', 'rating')
    list_filter = ('mfg_date', )
    actions = [change_rating]
```

Thanks for Reading